Guitar Learning Game

Project Synthesis Report

Comprehensive Analysis of Project Documentation

Generated: January 2026

1. Executive Summary

The Guitar Learning Game (GLG) is an educational software project designed to teach guitar and music theory through interactive gameplay. The application is built as a web-based HTML5 platform supporting browsers, tablets, smart boards, and desktop environments via PWA and Tauri. The project emphasizes education-first design, accuracy-driven music theory implementation, and non-inventive adherence to standard notation and engraving practices.

This synthesis consolidates information from 10+ project artifacts including design documents, technical specifications, build logs, conversation ledgers, and the authoritative system manual. The project has progressed through multiple build iterations from January 6-19, 2026, with an established instruction spine for tracking implementation progress.

2. Project Overview

2.1 Core Vision

The Guitar Learning Game serves as an educational tool that combines music theory instruction with engaging gameplay mechanics. Unlike traditional music learning applications, GLG integrates territory-based multiplayer mechanics, token systems, and connect-style bonus features while maintaining strict adherence to proper music notation standards. The design philosophy centers on three foundational principles: education-first approach where learning outcomes take precedence over entertainment mechanics; accuracy-driven implementation where all music theory, notation, and engaving follows established standards without simplification; and non-inventive methodology where the application extends rather than reinterprets existing musical conventions.

2.2 Platform Architecture

The technical architecture follows a modern web-first approach designed for broad accessibility. The primary platform targets web browsers including Chrome, Edge, and Safari on PCs, tablets, and smart boards. Progressive Web App capabilities enable offline functionality and desktop installation. Future desktop deployment is planned through Tauri framework integration. The application employs a unified input system supporting both touch and mouse interactions through Pointer Events API, ensuring seamless operation across device types without requiring platform-specific code paths.

2.3 Game Modes

Mode	Players	Scoring	Timer	Description
Single Player	1	Time-based	Core metric	Complete challenges in minimum time

Multiplayer	1-16	Competitive	Turn-based	Territory capture with tokens
Educational	1+	None	Optional	Instructor-led or self-guided learning
Exploration	1	None	Off	Free interaction and pattern discovery

Table 1: Game Mode Comparison

2.4 Learning Types

The application supports five primary learning content types that layer onto game modes. Single Notes focuses on individual pitch identification across fretboard, staff, and tablature. Intervals covers both absolute and relative interval recognition with support for simple and compound intervals. Chords includes quality recognition, voicing rules, inversions, and extensions. Scales supports linear, positional, and multi-position scale traversal. Arpeggios covers chord-derived traversal patterns with configurable direction options.

3. Core Surfaces

3.1 Fretboard

The fretboard visualization renders guitar neck positions with configurable fret ranges rather than defaulting to all 24 frets. This design decision reduces visual overload and improves learning focus. The fretboard supports configurable string inclusion/exclusion and preset ranges for common positions (5, 12, 24 frets). Interactive elements include direct highlighting for note selection without selection boxes, enharmonic space splitting where sharp notes occupy the left half and flats the right half, and territory ownership visualization for multiplayer sessions with vulnerability indicators.

3.2 Music Staff

The music staff implementation adheres to professional engraving standards, behaving identically to real sheet music. Critical requirements include correct clef generation per instrument covering treble, bass, and instrument-specific clefs; proper rendering of key signatures with sharps and flats, time signatures, and ledger lines above and below the staff; staff spacing accommodating whole to 32nd notes, triplets, beaming, and rests with correct rhythmic positioning rather than fixed vertical slots. The fundamental design principle states that if handed a printed score, the application should display identically.

3.3 Tablature (TAB)

Tablature operates as a separate rendering system using strings and numbers rather than dots. Alignment between staff and tab follows rhythmic precision rather than visual approximation. Input interaction uses direct highlighting without selection boxes, with explicit string specificity required. The tab system maintains independence from staff notation while preserving rhythmic correlation.

3.4 Note Rail

The Note Rail functions as both a visual reference and optional input interface. Supported display modes include note names, intervals, scale degrees, Roman Numeral Intervals, and plain numeric indexing (0-24). The rail defaults to prompt-first behavior with asked state ON and answered state OFF, avoiding implications of history persistence while maintaining prompt highlight activity.

3.5 Surface Controls

Control	Function
Prompt	Surface displays the question/challenge to player
Mark	Surface accepts player input
Persistence	Surface remains visible (replaces deprecated "Display")
SAM	Show After Miss - reveals correct answer briefly on error

Table 2: Surface Control Definitions

4. Multiplayer Systems

4.1 Territory Rules

The multiplayer mode implements a territory capture system where claimed spaces cannot be re-captured, preventing re-bonusing on previously owned territories. When notes become re-available through game mechanics, existing connections may break but prior bonuses never regenerate. This creates strategic depth while maintaining fairness across player rotations.

4.2 Connect-4 Bonus System

The game features a Connect-4 style bonus system where players earn rewards for creating sequences of four or more owned cells. Sequence detection operates in four directions: horizontal across frets, vertical across strings, diagonal up, and diagonal down. Enharmonic spaces provide unique strategic opportunities, counting as two independent values or one shared territory depending on game settings.

4.3 Token System

Tokens exist exclusively in multiplayer mode and are awarded when unique segments are completed. The awarding rule grants one token per NEW segment, not per note or extension, regardless of segment type (chords, scales, arpeggios). Tokens are awarded immediately upon completion rather than during bonus resolution, and difficulty settings do not affect token earning rates.

4.4 Stealing Mechanics

Stealing provides strategic options for players to capture opponent territory. Stealable conditions occur through two mechanisms: vulnerability condition where players miss sufficient turns causing notes to flash indicating stealability, and completion condition where all instances of a note are discovered making it stealable without flashing. After three consecutive misses on the same prompt, one owned cell for that prompt becomes vulnerable with CSS animation indicating the change.

4.5 Endgame / Blackout

The Blackout state triggers when the board becomes fully filled within the current pitch framework. When blackout occurs, all players receive one final turn with the triggering player going last. The triggering player continues until a miss occurs, with bonus use allowed during the final turn. No new territory scoring occurs after blackout is declared.

5. Project Status

5.1 Build Timeline

The project has progressed through multiple build iterations from January 6-19, 2026. Early builds (01_06_26 through 01_12_26) lacked the instruction spine documentation. The 01_13_26 builds introduced the DOCS folder and instruction spine files. Build 01_14_26_12 added LOCK_IN_TABLES. Build 01_16_26_98 achieved full instruction spine integrity (5/5). Subsequent builds have maintained documentation standards with occasional spine integrity variations during rapid development phases.

5.2 Completed Checklist Items

ID	Description	Status
GEG-001	Engage/End/New correctly control single match instance	Complete
GEG-002	Turn loop fully wired with bonus phase	Complete
GEG-003	Intermission phase displays and transitions	Complete
GEG-004	Bonus phase reachable and visible	Complete
GEG-010	Sequence detection in 4 directions	Complete
GEG-011	Segment scoring matches canon (4+ run)	Complete
GEG-012	Token awarding: 1 token per NEW segment	Complete
GEG-013	Steal token spend + resolution works	Complete
GEG-014	Last-chance phase works	Complete
GEG-015	Main Menu Settings: Token cap adjustable	Complete
GEG-016	Main Menu Settings: Starting tokens adjustable	Complete
GEG-020	Player card timer display	Complete
GEG-022	Active players only in leaderboard	Complete
GEG-080	Utility basics	Complete

Table 3: Completed Checklist Items

6. Documentation Architecture

6.1 Authority Hierarchy

The project maintains a strict authority hierarchy for resolving conflicts. Canon Design rules from Guitar_Education_Game_Comprehensive_Guide_Merged_Canon.docx take highest priority, followed by Canon Terms/Features from GLG_Term_Feature_Profiles_v2.docx. Historical fact is recorded in Master Timeline Ledger, with implementation evidence in project CHANGELOG excerpts and conversation_composite.json. Execution gates are managed through CHECKLIST.md, CANON_PROGRESS.md, DECISIONS_LOG.md, MATCH_SETTINGS_SPEC.md, THREAD_HANDOFF.md, and DOCS mappings.

6.2 Instruction Spine

The instruction spine consists of five core documents that must accompany any valid build. CHECKLIST.md defines completion gates and tracks task status. CANON_PROGRESS.md records development progress and build references. DECISIONS_LOG.md documents architectural decisions and their rationale. MATCH_SETTINGS_SPEC.md specifies match and session settings configuration. THREAD_HANDOFF.md provides context transfer between development sessions. Builds missing the instruction spine are considered invalid unless explicitly documented as legacy.

6.3 DOCS Index

The DOCS folder contains supplementary documentation including ADVANCED_MATCH_OPTIONS_UI_MAPPING.md for UI controls, CANONICAL_LAYERS_DIAGRAM.md for visual reference, DEPRECATED_TERMS.md enumerating obsolete terms and disallowed regressions, HOW_TO_PROPOSE_CHANGES.md for change control process, LOCK_IN_TABLES.md for locked definitions, PLATFORM_STRATEGY.md for deployment architecture, RUNTIME_CANON_GUARDS.md for validation rules, and START_SCREEN_UI_MAPPING.md for menu specifications.

7. Terminology Locks

The project has established locked terminology to prevent confusion and maintain consistency. The term 'Persistence' has replaced prior 'Display' usage in the current scheme, as boards remain always visible. Deprecated terms listed in DEPRECATED_TERMS.md include numbered 'modes', 'combined mode', and treating asked/answered/view as rules. These terms must not be reintroduced. The Note Rail defaults to prompt-first surface behavior with 'asked' state ON and 'answered' state OFF.

8. Technical Implementation

8.1 Architecture

The application follows an Engine/Renderer split architecture separating game logic from visual rendering. Canvas2D handles surface rendering for fretboard and staff/tab components. The system ensures deterministic repaint on each renderAll() pass to prevent blank surfaces after splash/visibility transitions. IndexedDB provides persistence for settings and leaderboards, with localStorage keys maintained for backward compatibility.

8.2 Testing Infrastructure

End-to-end testing uses Playwright running against built output via Vite preview on port 4173. This approach aligns with shipping configuration and reduces Windows/Node optional dependency flakiness. The testing workflow follows: build, preview, playwright execution. Development tools are gated behind a Dev/Test unlock mechanism to prevent accidental exposure in production builds.

9. Next Steps and Recommendations

9.1 Immediate Priorities

Based on NEW_DIRECTIONS.md and the current checkpoint at GEG-002, the recommended work queue follows a systematic approach. GEG-002 through GEG-004 should verify and finalize turn loop wiring, intermission phase, and bonus phase accessibility. GEG-005 addresses multiplayer player roster UI requirements. GEG-010 through GEG-016 covers scoring, token, and steal mechanics verification. Priority should be given to ensuring all checklist items marked complete are verified in the running build through runtime testing.

9.2 Documentation Maintenance

Any modifications must be recorded in CANON_PROGRESS.md with architectural changes justified in DECISIONS_LOG.md. Settings changes require updates to MATCH_SETTINGS_SPEC.md. Regressions to avoid include reintroducing deprecated terms, reverting menu flow to pre-Start-Setup state, removing required surfaces, breaking deterministic repaint behavior, and breaking dev-tool gating. The change control process should be followed for all modifications.

9.3 Feature Development

Future features should extend rather than reinterpret existing definitions. The strategic priority order remains: definitions, notation stabilization, education tools, and only then advanced gameplay. Explicitly parked future features include Guitar Pro file import, MusicXML interoperability, song-based lessons, external repository compatibility, analytical overlays, Circle of Fifths, and Circle of Thirds. These features should not be implemented until core functionality is stable and verified.

10. Conclusion

The Guitar Learning Game project demonstrates a well-structured approach to educational software development with comprehensive documentation, clear authority hierarchies, and systematic progress tracking. The established instruction spine and change control processes provide a solid foundation for continued development. The project has made significant progress on core gameplay mechanics, multiplayer systems, and UI infrastructure. Continued adherence to canon documentation and the authority hierarchy will ensure consistent implementation aligned with the educational vision and technical specifications defined in the authoritative documents.